Final Exam, Theory of Computation 2024

1 *(15 pts)* **Quick-fire round.** Consider the following statements.

1. Every language in $\mathbf{P}$ is regular.
2. Every finite language is regular.
3. Every language in $\mathbf{NP}$ is decidable.
4. Language $\{0^n 1^m : n \leq m\}$ is regular.
5. Language $\{w : \text{number of 1s in } w \text{ is divisible by } 2024\}$ is regular.
6. Language $\{\langle D \rangle : D \text{ is a DFA with } L(D) = \emptyset\}$ is in $\mathbf{P}$.
7. Language $\{\langle G \rangle : G \text{ is a graph with vertex cover of size } 2024\}$ is $\mathbf{NP}$-complete.
8. Suppose $A \subseteq B \subseteq C$ where A and C are decidable. Then B is recognisable.
9. If A and B are recognisable, then $A \cap \overline{B}$ is recognisable.
10. If $A \leq_m B$, then $A \leq_p B$.
11. If $A \leq_m B$, then $\overline{A} \leq_m \overline{B}$.
12. If $L \in \mathbf{NP}$, then $\overline{L} \in \mathbf{NP}$.
13. $\mathbf{P} = \mathbf{NP} \cap \mathbf{coNP}$.
14. 3-SAT $\leq_m$ HALT.
15. 2-SAT $\in \mathbf{P}$.

For each statement, write down one of

- **T** if the statement is known to be true.
- **F** if the statement is false *or not known to be true.* E.g., both $\mathbf{P} = \mathbf{NP}$ and $\mathbf{P} \neq \mathbf{NP}$ should be marked **F**.
- or leave your answer empty.

A correct **T**/**F** answer is worth $+1$ point, an incorrect answer is worth -1 point, and an empty answer is worth 0 points.

Solution:

1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.	13.	14.	15.
F	T	T	F	T	T	F	F	F	F	T	F	F	T	T

2 *(20 pts)* **Regular languages.**

2a Write down the statement of the Pumping Lemma. *(Only the statement, not its proof!)*

2b Let $\Sigma = \{a, b, c\}$. Show that the following language is regular:

$$\textsc{Double-}\Sigma \;=\; \{\, w \in \Sigma^* : \text{Every symbol in } \Sigma \text{ appears in } w \text{ at least twice} \,\}.$$

Solution:

2a If A is a regular language, then there is a number p (the pumping length) such that, for every string s in A of length at least p, there exists a division of s into three pieces, $s = xyz$ s.t.

- for each $i \geq 0$, $xy^iz \in A$
- $|y| \geq 1$, and
- $|xy| \leq p$.

Grading scheme:

- Correct statement - 7 pts
- If the statement is not correct - varies from case to case.

2b Consider language

$$\textsc{Double-}a \;=\; \{\, w \in \Sigma^* : a \text{ appears in } w \text{ at least twice}\},$$

and $\textsc{Double-}b$, $\textsc{Double-}c$ defined similarly. It is easy to see that $\textsc{Double-}\Sigma$ is the intersection of these three languages, so it is enough to prove that each of them is regular. Moreover, it is enough to do it for $\textsc{Double-}a$ – the proof for the other two languages is similar.

We will build a DFA that recognises $\textsc{Double-}a$:

- $Q = \{q_0, q_1, q_2\}$,
- $\delta(q_0, a) = q_1,\ \delta(q_0, b/c) = q_0;$
  $\delta(q_1, a) = q_2,\ \delta(q_1, b/c) = q_2;$
  $\delta(q_2.a/b/c) = q_2.$
- $F = \{q_2\}$.

Note that, after we finish reading w, we end up in q_2 if and only if w contains at least two a's, which precisely means that the DFA we have built recognises $\textsc{Double-}a$.

Grading scheme:

- Correct solution - 13 pts.
- Solution is incorrect but it has a good idea in it - 2 or more pts.

3 *(15 pts)* **NP-completeness I.** Show that the following language is **NP**-complete:

$$\text{DOUBLESAT} = \{ \langle\varphi\rangle : \varphi \text{ is a CNF that has at least two satisfying assignments} \}.$$

Solution: DOUBLESAT $\in$ **NP**: the verifier will take as arguments a formula φ and two assignments C_1 and C_2. We check that: 1) $C_1 \neq C_2$; 2) $\varphi(C_1) = 1$; 3) $\varphi(C_2) = 1$. This can all be done in polynomial time.

DOUBLESAT is **NP**-hard: we build a reduction from SAT in the following way. Given a formula ψ, we output a formula $\varphi := \psi \wedge (x \vee \neg x)$, where x is a new variable. If ψ had a satisfying assignment ρ, it can be extended to a satisfying assignment for φ in two ways: $\rho \cup \{x = 0\}$ and $\rho \cup \{x = 1\}$. Therefore, if $\psi \in$ SAT, then $\varphi \in$ DOUBLESAT. On the other hand, if $\varphi \in$ DOUBLESAT, we can take a satisfying assignment for φ and project it onto variables of ψ (that is, discard value of x), and the result will be a satisfying assignment for ψ.

Grading scheme: up to 7 points for **NP**-membership and up to 8 points for **NP**-hardness.

4 *(15 pts)* **NP-completeness II.** Suppose $\mathcal{S} = \{S_1, \ldots, S_n\}$ is a family of subsets of $\{1, 2, \ldots, n\}$. We say that k distinct sets $S_{i_1}, \ldots, S_{i_k} \in \mathcal{S}$ form a *packing of size k* if the sets are pairwise disjoint, that is, $S_{i_j} \cap S_{i_{j'}} = \emptyset$ for $j \neq j'$. Show that the following problem is **NP**-complete:

$$\text{SETPACKING} = \{ \langle \mathcal{S}, k\rangle : \text{Family } \mathcal{S} \text{ contains a packing of size } k \}.$$

Solution: To prove SETPACKING $\in$ **NP**, we give the verifier as follows: Given a set of indices $I = \{i_1, \ldots, i_k\}$, the verifier checks that whether for every $1 \leq j < j' \leq [k]$, $S_{i_j} \cap S_{i'_j} = \emptyset$. If yes, the verifier accpets, rejects otherwise. It is clear that such a verifier runs in polynomial time.

Next, we prove SETPACKING is **NP**-hard by reducing INDSET to SETPACKING. Given an INDSET instance $\langle G, k\rangle$, we construct a SETPACKING instance $f(\langle G, k\rangle) = \langle \mathcal{S}, k'\rangle$ as follows: Let let $k' := k$ and m denote the number of edges of G. We assign each edge e with a unique label $\ell_e \in [m]$. For each vertex v_i, we construct a subset $S_i \subseteq [m]$ which consist of the labels of all the edges incient to v_i.

Finally, let us prove the correctness of our reduction. Consider the following four statements:

(i) $\{v_{i_1}, \ldots, v_{i_k}\}$ is an indepdent set of G.

(ii) For every edge e, at most one endpoint is in $\{v_{i_1}, \ldots, v_{i_k}\}$.

(iii) For every edge e, the corresponding label is contained in at most one of $S_{i_1}, \ldots, S_{i_k}$.

(iv) $S_{i_1}, \ldots, S_{i_k}$ is a packing.

By the definition of independent set, (i)$\equiv$(ii). By the defintion of packing, (iii)$\equiv$(iv). It also follows from our reduction that (ii)$\equiv$(iii). Thus (i)$\equiv$(iv), which in turn implies that $\langle G, k\rangle \in$ INDSET if and only if $\langle \mathcal{S}, k'\rangle \in$ SETPACKING.

Grading scheme:

- **NP**-membership - 7 pts

- **NP**-hardness - 8 pts.

5 *(20 pts)* **Undecidability.** Consider the language

$$\textsc{DoubleTM} \;=\; \{\, \langle M \rangle : M \text{ is a TM that accepts at least two inputs, i.e., } |L(M)| \geq 2 \,\}.$$

Classify DoubleTM as one of (i) decidable, (ii) undecidable but recognisable, (iii) unrecognisable. Justify your answer with a proof. (*Note: You are not allowed to use Rice's theorem as mentioned in Homework 2! Use only concepts introduced in class/exercises.*)

Solution: We will prove that the language DoubleTM is **(ii) undecidable but recognizable**. We will first construct a recognizer R for DoubleTM. Let $\{w_i\}_{i=1}^{\infty}$ be some computable enumeration of the strings in the input alphabet Σ. We define R as follows:

$R =$ "On input x:

1. **If** x is not a valid encoding of a TM, **reject**.
2. Otherwise, for $x = \langle M \rangle$, do the following:

   for $i \geq 1$:

      Simulate M on $w_1, w_2, \ldots, w_i$ for i steps.
      If M's computation results in an accept state for at least 2 of the strings $w_1, w_2, \ldots, w_i$, **break** out of the loop and **accept** $\langle M \rangle$."

Clearly, R accepts $\langle M \rangle$ if and only if $|L(M)| \geq 2$. Hence, DoubleTM is recognizable.

Now, we proceed to prove that DoubleTM in undecidable through a reduction from the undecidable language Halt. That is, we will prove that Halt $\leq_m$ DoubleTM by constructing a computable function which maps the pair $\langle M, w \rangle$ to a TM $\langle M_w \rangle$ such that $|L(M_w)| \geq 2 \iff M(w)$ halts. Without further ado,

$M_w =$ "On input x:

1. **If** $x = w \cdot 0$ (the concatenation of w and 0), **accept**.
2. Otherwise, **simulate** M on w. **If** $M(w)$ halts, **accept** w.

Now, if $\langle M, w \rangle \in$ Halt, then $L(M_w) = \{w, w \cdot 0\}$, so $\langle M_w \rangle \in$ DoubleTM. And if $M(w)$ loops forever, $L(M_w) = \{w \cdot 0\}$, so $\langle M_w \rangle \notin$ DoubleTM. Hence, the reduction is valid, and since Halt is undecidable, so is DoubleTM.

Grading Scheme. Note that correct solutions which do not adhere to the official solutions (or are not captured completely by the grading rubric) will receive full points. In some cases, points might be awarded for interesting approaches to the problem which do not lead to a complete solution.

- *2 pts.* for answering correctly with "(ii) recognizable but undecidable."
- *9 pts.* for proving DoubleTM is recognizable:

  - *3 pts.* for a wrong formal definition of a verifier that contains some hint of a correct approach but requires considerable modifications to work;
  - *6 pts.* for a slightly wrong formal definition of a verifier (e.g. the student did not account for the fact that when simulating $M(w)$, the simulation might loop forever);
  - *9 pts.* for an essentially correct formal definition of a verifier.

- *9 pts.* for proving DoubleTM is undecidable:
  - *3 pts.* for an attempt at a reduction which requires substantial amount of work to be fixed;
  - *6 pts.* for a slightly wrong reduction that can easily be fixed;
  - *9 pts.* for an essentially correct reduction to DoubleTM from an undecidable language.

6 *(15 pts)* **Doubly accepting NFA.** Let N be an NFA. The usual definition of acceptance is the existence of at least one accepting path. Change this definition as follows: N accepts the input word w iff there are at least two accepting paths. Denote by $L_2(N)$ the set of strings that admit at least two accepting paths. Prove that $L_2(N)$ is regular.

Solution: Let us convert doubly accepting NFA (daNFA) into a DFA following the standard NFA to DFA conversion. Recall that the states of the DFA in the standard conversion corresponded to $\{0,1\}^Q$, that is, binary vectors of length $|Q|$ encoding a subset of Q. In our case the states will correspond to $\{0,1,2\}^Q$, that is, ternary vectors of length $|Q|$. Such a vector will encode *how many paths* (0: zero, 1: one, 2: at least two) the current word can reach each of the daNFA states. The final states of the DFA are all that have at least 2 paths in any of the accepting states in daNFA.

Formally suppose that $A \coloneqq \langle Q, \Sigma, \delta, q_0, F\rangle$ is the initial doubly accepting NFA. Then define a DFA $D \coloneqq \langle \{0,1,2\}^Q, \Sigma, \delta', q_0', F'\rangle$ with

- For every $q \in Q$ define $(q_0')_q \coloneqq \begin{cases} 1 & \text{if } q = q_0 \\ 0 & \text{otherwise} \end{cases}$.
- For every $r \in \{0,1,2\}^Q$ and $\alpha \in \Sigma$ let $\delta'(r,\alpha) = \{\min\{\sum_{t\in Q\colon \delta(t,\alpha)\ni q} r_t, 2\}\}_{q\in Q}$.
- $F' \coloneqq \{r \in \{0,1,2\}^Q \mid \exists q \in F\colon r_q = 2\}$

For a word $w = \alpha_1, \ldots, \alpha_k$ we can chow by induction on the length of the word that for $r = \delta'(q_0', w)$, the number $r_q = \min\{2, \text{number of paths in } A \text{ from } q_0 \text{ to } q\}$. Hence w is accepted by D if and only if there exists $q \in F$ with at least two accepting paths in A. This concludes the proof.

Grading Scheme: The following are applied exclusively.

- *2 pts.* Showing that a regular language can be recognised by a daNFA.
- *2 pts.* Alternative incorrect solutions.

- *3 pts.* Suggesting adapting NFA to DFA conversion to the current problem.
- *7 pts.* Solving the problem with another definition "daNFA accepts a word s if there are two distinct accepting *states* to which there is an accepting path corresponding to s". Such solutions are, for instance
  - The standard conversion NFA to DFA conversion and defining the final states as $\{S \subseteq Q \mid |S \cap F| \geq 2\}$ where Q is the set of stated of daNFA and F is the set of its final states.
  - Making $|F|$ copies of A leaving only one accepting state in each and then multiplying them.
- *15 pts.* Correct solution.
  - *(-5 pts)* If there is no correct proof.